DEPI GRADUATION PROJECT

PortfolioGenie

AI-Powered Developer Portfolio Builder

SYSTEM ANALYSIS AND DESIGN

Engineering Documentation | Initial Design Baseline

DOCUMENT STATUS	SOURCE BASIS	VERSION
Prepared from the three supplied project-phase documents	Planning & Management · Literature Review · Requirements Gathering	1.0 — Draft for mentor/team validation

Prepared for DEPI project documentation. Team member names, mentor name, and official submission metadata were not supplied and are intentionally omitted.

Document Contents

- 1. Executive Summary
- 2. Project Context and Scope
- 3. Stakeholders and User Profile
- 4. System Analysis
- 5. Functional Requirements
- 6. Non-Functional Requirements
- 7. Use Cases and User Stories
- 8. System Design
- 9. Architecture and Workflow Diagrams
- 10. Data and Interface Design
- 11. Security, Privacy, and Quality
- 12. Risks, Assumptions, and Open Decisions
- 13. Verification and Acceptance Matrix
- 14. Conclusion
- Appendix A. Source Traceability
- Appendix B. References

Document note: This report is a design baseline derived from the supplied phase documents. Items marked proposed or open are not represented as already implemented.

1. Executive Summary

PortfolioGenie is a web-based platform intended to help junior developers and students create a professional developer portfolio through a guided multi-step form. The user enters profile information, technical skills, and project details; navigates between an editor and a live preview; and may request AI-assisted improvements to biography or project-description text. The user retains control over AI output and can accept, edit, or discard suggestions.

The design emphasis is a focused developer portfolio workflow—not a general website builder, recruitment platform, social network, or payment product. The supplied documents confirm React, TypeScript, React Hooks, Custom Hooks, React Router, UI/UX practices, prompt engineering, and clean modular code as the named technology/design direction. They do not confirm a mandatory backend, database, authentication system, or hosting provider.

Design boundary

The architecture and diagrams in this report model the confirmed client-side workflow and show the AI service as an optional external integration. Persistence, publishing, and service-provider details remain open decisions. Any architecture beyond the confirmed brief is explicitly labeled as a proposal.

1.1 Document objectives

This document translates the initial project concept and requirements into a structured analysis and design baseline. It supports implementation planning, mentor review, and later test-case development. It is not evidence that the described system has already been implemented or deployed.

2. Project Context and Scope

2.1 Problem statement

Junior developers often need a clear, professional way to present skills and projects to recruiters, but building a portfolio from scratch can require time, design experience, and front-end development knowledge. PortfolioGenie addresses this problem with guided data entry, reusable layouts, live preview, and optional AI writing assistance.

2.2 Aim and objectives

- Provide a simple workflow for creating a developer portfolio.
- Allow users to enter and update profile details, technical skills, and project information.
- Organize portfolio creation as a multi-step form.
- Show live preview as users edit supported data.
- Use prompt engineering with an AI service to suggest clearer biographies and project descriptions.
- Follow responsive design and accessibility principles.
- Maintain clean, modular, maintainable code.

2.3 Scope

IN SCOPE	OUT OF SCOPE FOR INITIAL VERSION
Landing page explaining the platform Guided portfolio creation form Profile, skills, and project sections Portfolio editor and live-preview navigation AI-assisted biography/project text improvement Responsive UI and basic accessibility	Full job application or recruitment system Social network or public discussion platform Complex payment/subscription features Guaranteed automatic publishing to every hosting provider

2.4 Proposed technology baseline

Area	Confirmed in supplied planning document	Design purpose
Frontend	React	Reusable UI components and application pages
Language	TypeScript	Static typing and maintainability
React state/effects	useState, useEffect	Manage component state and side effects
Reusable logic	Custom Hooks	Encapsulate multi-step form logic
Navigation	React Router	Editor/preview navigation
AI	Prompt engineering + AI service (provider TBD)	Improve biography and project descriptions; SEO-oriented wording noted in plan
UI/UX	Design thinking, personas, wireframes	User-centered accessible experience

Backend, database, authentication, and hosting are not confirmed by the supplied planning or requirements documents. They must be decided only after review of the complete official project brief and mentor instructions.

3. Stakeholders and User Profile

3.1 Stakeholder groups

Stakeholder	Interest / role	Design consideration
Junior developer / student (primary user)	Create and customize a professional portfolio; present skills/projects clearly; may seek internship or junior roles.	Guided steps, simple labels, editing, preview, accessible controls, help with text.
Recruiter / hiring manager (secondary stakeholder)	View a portfolio shared by its creator.	Clear content hierarchy and readable portfolio output; sharing method depends on implementation.
Project team	Build, test, document, and hand over the project.	Modular responsibilities, consistent requirements, clear open decisions.
Mentor and DEPI evaluators	Review scope, progress, and final project package.	Traceable requirements, diagrams, test evidence, and complete documentation.

3.2 Primary persona

Attribute	Persona description
Profile	Student, graduate, or early-career developer seeking internships or junior roles.
Goals	Present skills and projects clearly and create a shareable professional portfolio.
Needs	Simple instructions, guided data entry, customization, live preview, and help improving text.
Pain points	Limited design experience, uncertainty about what to write, and difficulty organizing project information.
Accessibility	Readable text, keyboard-friendly controls, clear labels, responsive layouts.

4. System Analysis

4.1 Proposed system overview

PortfolioGenie provides a guided sequence for collecting portfolio content and presenting it through an editor/preview workflow. The current-session portfolio data is used to render the preview. The AI-assisted writing flow is optional and depends on the AI feature being configured and approved. Persistent saving and publishing are not defined in the supplied requirements and therefore are not included as confirmed system behavior.

4.2 System boundary

The confirmed boundary includes the user-facing web application, portfolio form/editor, preview rendering/navigation, and optional interaction with an external AI service. The system boundary does not include a confirmed server, database, identity provider, or deployment/hosting platform.

4.3 Business rules

- Portfolio content is entered by the user through a guided workflow.
- Profile information includes name, role, biography, and contact/social links.
- Skill entries can be added, edited, and removed.
- Project entries can be added, edited, and removed; the described project fields include title, description, technologies, and optional repository/demo links.
- Required or invalid fields must produce understandable feedback.
- Changes to supported fields should be reflected in the preview without re-entering data.
- AI output is a suggestion: the user chooses to accept, edit, or discard it.
- If an AI request fails, the original text must be preserved.
- Portfolio data is available for preview in the current session; persistent saving depends on the approved architecture.
- Sharing/publishing is limited to a format supported by the eventual implementation.

4.4 System constraints

- React Hooks (useState and useEffect), Custom Hooks for multi-step form logic, React Router, UI/UX practices, AI prompt engineering, and clean code are explicitly named in the supplied brief.
- AI provider, API access, cost/limits, and data handling must be decided before enabling the AI feature.
- Saving and publishing method must be confirmed before implementation.
- The complete official DEPI brief and mentor instructions must be checked for additional mandatory requirements.
- Do not expose sensitive AI API keys in frontend source code.

4.5 System context diagram

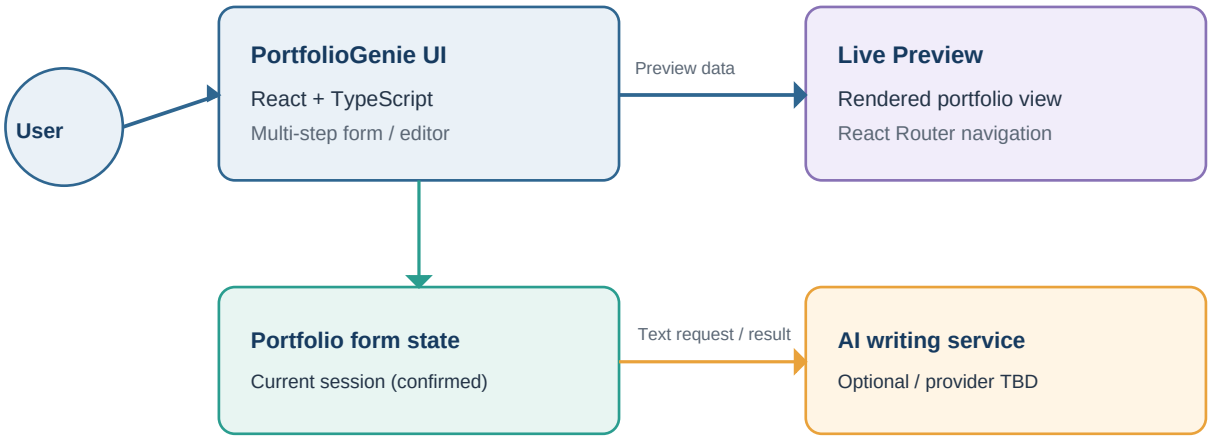


Figure 1. System context and confirmed boundary. AI is shown as an optional external service. The current-session state and live preview are confirmed; persistence and hosting are intentionally not shown as established components.

5. Functional Requirements

The following requirements preserve the IDs and priority levels from the supplied Requirements Gathering document. Additional design interpretations are labeled as notes rather than silently changing the source requirements.

ID	Requirement	Priority	Acceptance criteria
FR-01	Enter basic profile information (name, role, biography, contact/social links).	Must	Entered values appear correctly in portfolio preview.
FR-02	Add, edit, and remove technical skills.	Must	User can create, update, and remove a skill entry.
FR-03	Add, edit, and remove project entries.	Must	Project can include title, description, technologies, and optional repository/demo links.
FR-04	Present portfolio creation as a multi-step form.	Must	User can move between steps and receives clear validation feedback.
FR-05	Provide navigation between editor and live preview using React Router.	Must	Editor and preview routes can be opened and navigated between.
FR-06	Update preview when relevant portfolio data changes.	Must	Supported field changes appear in preview without re-entering data.
FR-07	Offer AI-assisted biography and project-description improvement.	Should; subject to approved brief and API access	User requests suggestion, reviews it, and chooses whether to use it.
FR-08	Provide feedback for missing or invalid required fields.	Must	Invalid input is identified with understandable messages.
FR-09	Provide a way to view or share completed portfolio.	Should; publishing method to be confirmed	Portfolio can be presented in a shareable format supported by implementation.

5.1 Requirement interpretation

FR-09 is not equivalent to automatic public hosting. The supplied requirement explicitly leaves the publishing method open and requires only a shareable format supported by the implementation. FR-07 remains a Should requirement and is conditional on the approved brief and API access.

6. Non-Functional Requirements

ID	Quality attribute	Acceptance guidance from source
NFR-01	Responsive design	Core flows remain usable on mobile, tablet, and desktop sizes.
NFR-02	Accessibility	Controls have labels, keyboard navigation is supported, and text contrast is considered.
NFR-03	Usability	Steps, navigation, validation, and actions are understandable to first-time users.
NFR-04	Maintainability	Components and reusable hooks have clear responsibilities and consistent naming.
NFR-05	Performance	Typing and preview updates should feel responsive under normal use.
NFR-06	Reliability	Clear error state is shown if an optional AI request fails.
NFR-07	Privacy and security	Sensitive API keys must not be exposed in frontend source; data handling must be documented if backend is added.
NFR-08	Compatibility	Core workflow should be tested in current major browsers.

6.1 Proposed verification targets (not source-defined thresholds)

The source documents provide qualitative acceptance guidance, not numeric performance or browser-version thresholds. The team should agree on measurable targets with the mentor before testing. Until then, test the complete workflow at representative mobile, tablet, and desktop widths; keyboard-only navigation; required-field errors; preview synchronization; and AI failure handling.

7. Use Cases and User Stories

7.1 Use Case Diagram

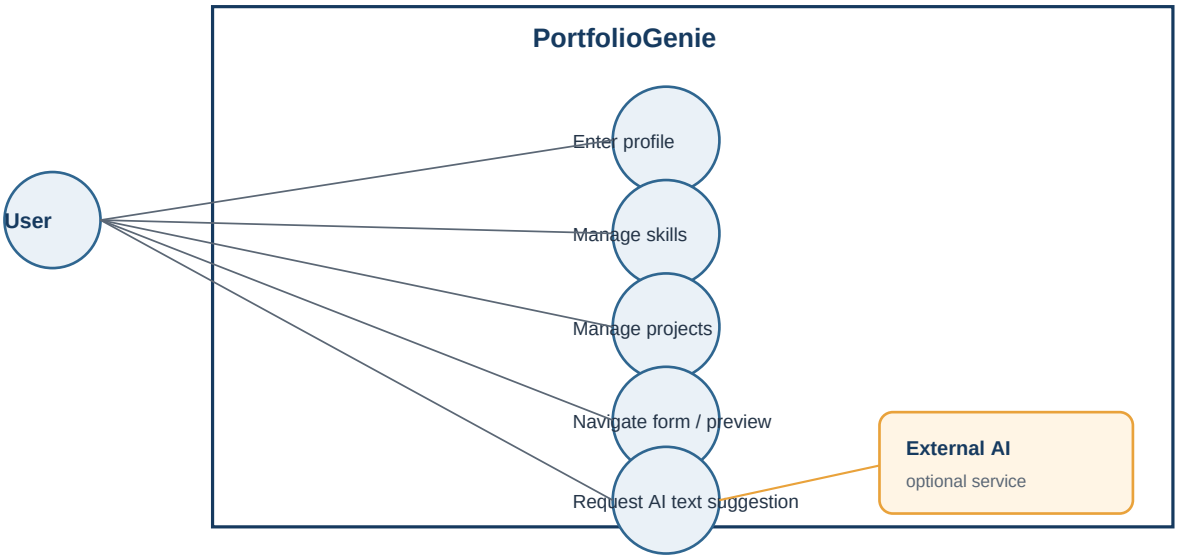


Figure 2. Use cases based on the two supplied use-case descriptions and functional requirements. External AI is involved only in the optional text-improvement use case.

7.2 UC-01 — Create a Portfolio

Field	Description
Actor	Junior Developer
Precondition	Application is open.
Main flow	User starts the form, enters profile information, adds skills and projects, reviews information, and opens the preview.
Alternative flow	If required information is missing, the system identifies the relevant field and allows the user to correct it.
Postcondition	Portfolio data is available for preview in the current application session. Persistent saving depends on final approved architecture.

7.3 UC-02 — Improve Text with AI

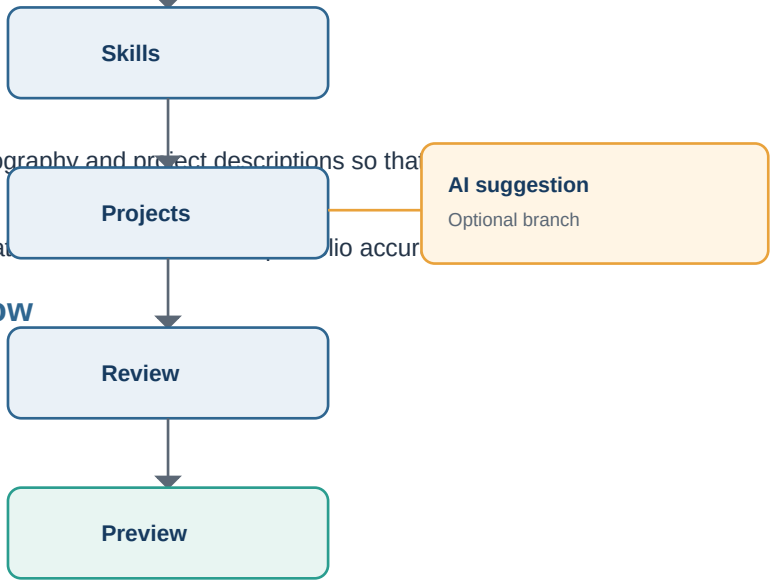
Field	Description
Actor	Junior Developer
Precondition	User has entered biography or project description and AI feature is configured.
Main flow	User requests a suggestion; system sends text with a defined prompt to AI service; result is displayed for review.
Alternative flow	If request fails or service is unavailable, system displays an error and preserves original text.
Postcondition	User chooses whether to accept, edit, or discard the suggestion.

7.4 User stories

- As a junior developer, I want to enter my information step by step so that creating a portfolio feels manageable.
- As a junior developer, I want to add projects and technologies so that recruiters can understand my practical skills.
- As a user, I want to see a live preview so that I can identify and correct presentation issues.

- As a user, I want AI suggestions for my biography and project descriptions so that I can present my experience more clearly.
- As a user, I want to edit or reject AI-generated content before it appears in my portfolio accurately.

7.5 Portfolio creation activity flow



Missing required field → show validation feedback → user corrects field

Figure 3. Proposed flow derived from UC-01. The AI suggestion is an optional branch; validation feedback returns the user to the relevant field.

8. System Design

8.1 Design principles

- Guided workflow: break portfolio creation into ordered steps.
- User control: preserve user-authored content and let the user decide whether to use AI output.
- Preview-driven feedback: reflect supported field changes in the live preview.
- Modularity: assign clear responsibilities to components and reusable hooks.
- Responsive and accessible interaction: design for common screen sizes, labels, keyboard access, and readable contrast.
- Scope integrity: do not introduce backend, database, login, or universal publishing without approval.

8.2 Logical component model (proposed)

Component / module	Responsibility	Status
Landing Page	Explain the platform and guide users to portfolio creation.	In scope; detailed implementation TBD
Portfolio Form / Step Controller	Manage step navigation and validation feedback.	Multi-step form confirmed; component name proposed
Profile Section	Capture name, role, biography, contact/social links.	Fields confirmed
Skills Section	Add, edit, remove technical skills.	Behavior confirmed
Projects Section	Add, edit, remove project records and supported project fields.	Behavior confirmed
Portfolio State / Custom Hook	Keep entered data available across form steps and preview in current session.	State behavior inferred from requirements; persistence TBD
Editor / Preview Router	Navigate between editor and live preview using React Router.	React Router confirmed
Preview Renderer	Render supported portfolio data and reflect changes.	Live preview confirmed
AI Writing Adapter	Send selected text and prompt to configured AI service; show suggestion and failure state.	Optional; provider/API TBD

8.3 State and data handling

The requirements state that portfolio data is available for preview in the current application session. A suitable client-side design is to keep profile, skills, and projects in shared form state managed through React state and reusable custom hooks, then pass the supported data to the preview renderer. This is a proposed implementation pattern, not a confirmed code structure. Durable storage, account-based retrieval, or cloud synchronization are not defined.

8.4 AI interaction design

The AI feature should be invoked only after the user enters biography or project-description text and requests help. The application sends the text with a defined prompt to the configured AI service, displays the returned suggestion, and gives the user accept/edit/discard choices. If the service fails, the application displays an understandable error and preserves the original text. The AI provider, transport, server-side proxy, limits, and data-retention policy remain open decisions.

9. Architecture and Workflow Diagrams

9.1 AI-assisted writing sequence (logical)

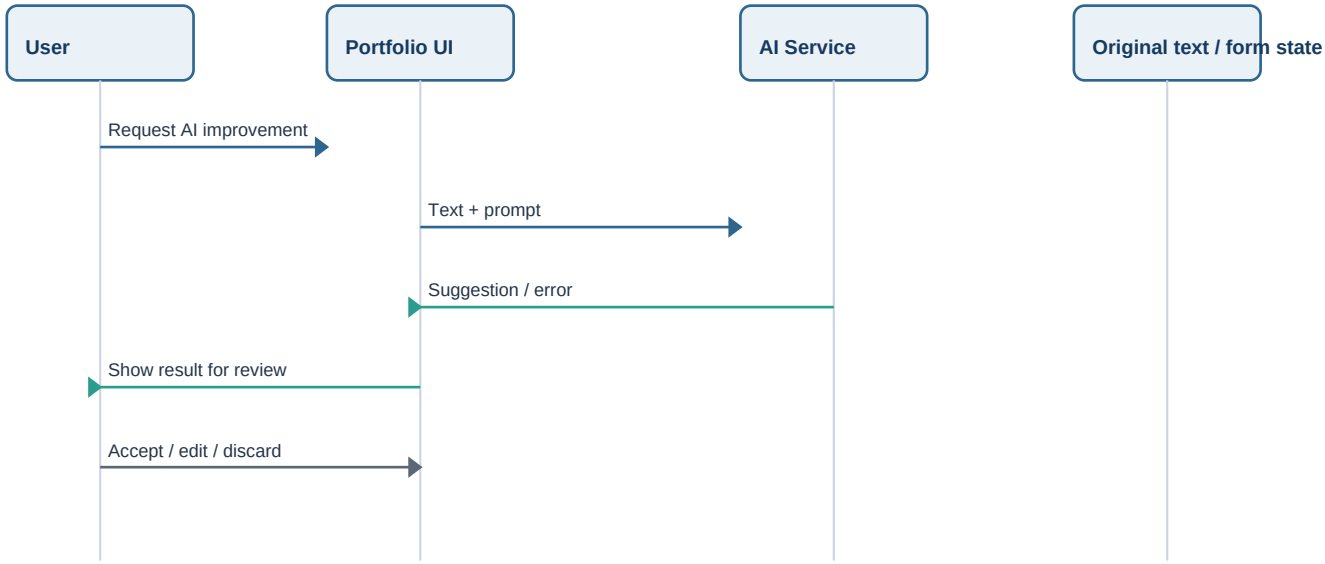


Figure 4. Logical AI interaction. This diagram reflects UC-02; exact API transport and server architecture are not specified.

9.2 Error and recovery behavior

Condition	Expected behavior	Requirement basis
Missing/invalid required field	Identify field and show understandable validation feedback; allow correction.	FR-04, FR-08, UC-01
AI service fails or is unavailable	Show clear error state and preserve original text.	NFR-06, UC-02
AI suggestion returned	Display for review; user may accept, edit, or discard.	FR-07, UC-02
Preview data changes	Update supported preview fields without re-entering data.	FR-06

9.3 Deployment diagram — intentionally deferred

A deployment diagram would require confirmed runtime, hosting, backend, and AI-service topology. The supplied documents do not identify those details. Therefore, this report does not present a fabricated deployment diagram. Add one after the team confirms the actual deployment plan.

10. Data and Interface Design

10.1 Portfolio data model (conceptual)

The following conceptual data groups are directly supported by the requirements. They describe information held for the portfolio workflow, not a confirmed database schema.

Data group	Fields / content	Operations / constraints
Profile	Name, role, biography, contact/social links	Enter/update; requiredness of each individual field must be confirmed in validation rules.
Skill	Technical skill entry; exact attribute schema not specified	Add, edit, remove.
Project	Title, description, technologies, optional repository/demo links	Add, edit, remove; optional links remain optional.
AI suggestion	Original biography/project text and generated suggestion	Suggestion is reviewed; original text is preserved on failure; user decides whether to apply.

10.2 Conceptual relationship diagram

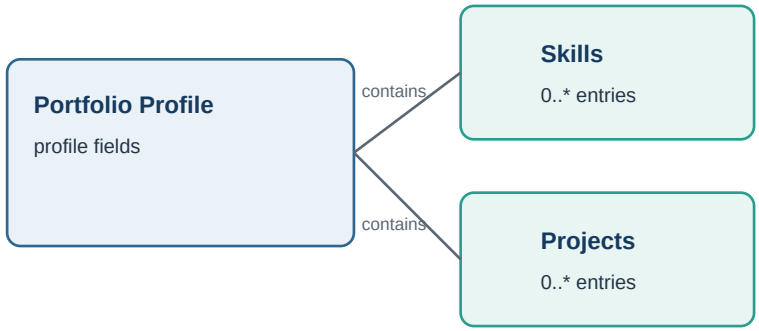


Figure 5. Conceptual in-session data grouping. Cardinality indicates zero or more skill/project entries for a profile, based on add/remove requirements. This is not a MongoDB or relational database ERD; no database is confirmed.

10.3 API design status

No application API endpoints are specified in the supplied project-phase documents. Because a mandatory backend is not confirmed, this report does not invent REST routes or request/response contracts. If a backend is approved, define endpoint contracts after confirming persistence, AI proxying, authentication, and sharing requirements.

10.4 Navigation model

React Router is named in the planning document for navigation between editor and live preview. Exact route paths are not supplied. Route names such as /editor and /preview may be adopted by the team, but should be treated as proposed identifiers until confirmed in implementation.

11. Security, Privacy, and Quality

11.1 Security and privacy design considerations

Concern	Design consideration	Status
AI API key	Do not expose sensitive API keys in frontend source. Choose a secure integration approach before enabling AI.	Explicit NFR-07; architecture TBD
User text sent to AI	Document what text is transmitted, why, and how it is handled; minimize unnecessary data.	Proposed consideration; policy TBD
Input validation	Validate required fields and provide clear feedback; validate optional URLs/fields according to agreed rules.	Required validation confirmed; detailed rules TBD
AI output	Treat output as suggestion, not authoritative user data; user must review and decide.	Confirmed user-control requirement
Error handling	Preserve original text and show a clear failure state when AI is unavailable.	Confirmed UC-02 / NFR-06
Persistence	If backend/storage is added, document data handling and access controls before implementation.	Conditional; backend not confirmed

11.2 Quality strategy

- Usability: validate that first-time users understand each step and can correct errors.
- Accessibility: check labels, keyboard focus/order, and text contrast.
- Responsive behavior: verify the core flow on mobile, tablet, and desktop.
- Performance: observe typing and preview updates under normal use; agree numeric thresholds with mentor.
- Reliability: test AI success, failure, unavailable service, and preservation of original text.
- Maintainability: review component boundaries, custom-hook responsibilities, naming, and code documentation.
- Compatibility: run the core workflow in current major browsers as required by NFR-08.

11.3 Testing approach

Use the acceptance criteria in the requirements document as the basis for test cases. Prioritize Must requirements before Should requirements. The table in Chapter 13 maps each confirmed requirement to a practical verification action; numeric thresholds and exact browser versions must be agreed with the mentor.

12. Risks, Assumptions, and Open Decisions

12.1 Risks and mitigations from planning

Risk	Potential impact	Mitigation from source
Unclear requirements	Rework and missed features	Confirm scope with project brief and mentor before implementation.
AI API limits or downtime	AI assistance unavailable	Show clear errors and allow manual editing without AI.
Responsive layout issues	Poor mobile experience	Test common viewport widths throughout development.
Overly large scope	Late delivery	Prioritize core portfolio workflow before optional enhancements.
Poor input validation	Incorrect/incomplete portfolio data	Validate required fields and provide clear feedback.

12.2 Assumptions and open decisions

ID	Decision	Current evidence	Action
OD-01	Backend requirement	Not confirmed in supplied brief.	Mentor/team to confirm whether a backend is required.
OD-02	Database / persistence	Not confirmed; portfolio data is available in current session only in UC-01.	Confirm save/export/storage behavior.
OD-03	Authentication	Not confirmed.	Do not add login as mandatory without approval.
OD-04	Publishing / sharing	FR-09 requires a shareable format; exact method is open.	Confirm whether export, static page, or hosted link is expected.
OD-05	AI provider and API access	Provider, limits, cost, and data handling are open.	Select provider and define secure integration/data policy.
OD-06	Template count / design tool	Evaluator expectation not specified.	Confirm required number of layouts and design artifacts.
OD-07	Testing format / standard	Evaluator expectation not specified.	Confirm required test report format and documentation standard.
OD-08	Team/mentor metadata	Names not present in supplied files.	Add verified names and roles before official submission.

12.3 Decision gate

Do not finalize deployment, database, authentication, or API diagrams until OD-01 through OD-05 are resolved. The team can proceed with client-side workflow and UI design using the confirmed scope while these decisions are reviewed.

13. Verification and Acceptance Matrix

The verification actions below operationalize the acceptance criteria supplied in Requirements Gathering. They are test suggestions, not completed test results.

Req ID	Verification action	Result
FR-01	Enter name, role, biography, contact/social links; verify preview values match.	Not executed — implementation evidence required
FR-02	Create a skill, edit it, then remove it; verify each operation.	Not executed — implementation evidence required
FR-03	Create project with title/description/technologies and optional links; edit and remove it.	Not executed — implementation evidence required
FR-04	Navigate across form steps; trigger missing required field; verify clear feedback.	Not executed — implementation evidence required
FR-05	Open editor and preview routes and navigate between them.	Not executed — implementation evidence required
FR-06	Change supported field and verify preview updates without re-entering data.	Not executed — implementation evidence required
FR-07	Request AI suggestion; verify review and accept/edit/discard choices (when AI configured).	Not executed — implementation evidence required
FR-08	Submit invalid/missing required input; verify field-level understandable message.	Not executed — implementation evidence required
FR-09	Verify completed portfolio can be presented in the shareable format actually implemented.	Not executed — implementation evidence required
NFR-01	Exercise core workflow at agreed mobile, tablet, and desktop viewport sizes.	Not executed — implementation evidence required
NFR-02	Check labels, keyboard navigation, focus visibility, and contrast.	Not executed — implementation evidence required
NFR-03	Ask a first-time user to complete core workflow; observe navigation and action clarity.	Not executed — implementation evidence required
NFR-04	Review component and custom-hook responsibilities and naming.	Not executed — implementation evidence required
NFR-05	Observe input and preview responsiveness under normal usage; define threshold with mentor.	Not executed — implementation evidence required
NFR-06	Simulate AI failure/unavailability; verify error state and original text preservation.	Not executed — implementation evidence required
NFR-07	Inspect frontend build/source for exposed AI secrets; document data handling if backend added.	Not executed — implementation evidence required
NFR-08	Run core workflow in agreed current major browsers.	Not executed — implementation evidence required

13.1 Definition of readiness

A requirement is ready for acceptance when its stated acceptance criterion is demonstrated, relevant errors are handled, and the team records evidence (test case, screenshot, or test report). AI-related checks are conditional on an approved and configured AI integration. FR-09 is conditional on the actual sharing method selected by the team.

14. Conclusion

This System Analysis & Design baseline organizes PortfolioGenie around its confirmed core: guided portfolio entry, profile/skills/projects management, editor-to-preview navigation, responsive and accessible interaction, and optional AI-assisted writing with user review. The analysis retains the initial project's junior-developer focus and avoids expanding it into recruitment, social networking, payments, or guaranteed hosting.

The proposed component and workflow diagrams provide a design direction for implementation. The conceptual data model is intentionally not presented as a database schema, because persistence technology is not confirmed. The team should resolve the open decisions, validate the report with the mentor and full official DEPI brief, then update diagrams and acceptance tests to reflect the approved architecture.

Recommended next actions

- Confirm mandatory project brief details with mentor.
- Decide persistence and sharing/publishing method.
- Choose AI provider and secure integration/data-handling approach.
- Create wireframes and final UI flow for the confirmed scope.
- Implement core Must requirements before optional AI/publishing enhancements.
- Run the acceptance matrix and attach real test evidence.
- Replace draft status with final version and add verified team/mentor details.

Appendix A. Source Traceability

Report content	Source document / section
Project overview, problem, objectives, scope, stakeholders	Project Planning & Management, Sections 1–5 (pages 1–2).
Technology baseline, WBS, timeline, risks, success criteria, assumptions	Project Planning & Management, Sections 6–11 (pages 2–4).
Existing solution review and opportunity	Literature Review, Sections 1–7 (pages 1–3).
Functional and non-functional requirements	Requirements Gathering, Sections 3–4 (pages 1–2).
Use cases, user stories, constraints, open questions, validation	Requirements Gathering, Sections 5–9 (pages 2–4).

Appendix B. References

The following references are copied from the supplied Literature Review. Product features and plan limits may change; the source document recommends checking official product documentation again before final submission.

GitHub Pages. Official website.
https://pages.github.com/

Wix. Official website.
https://www.wix.com/

Framer. Official website.
https://www.framer.com/

W3C Web Accessibility Initiative. Introduction to Web Accessibility.
https://www.w3.org/WAI/fundamentals/accessibility-intro/

Google Search Central. SEO Starter Guide.
https://developers.google.com/search/docs/fundamentals/seo-starter-guide

Source note: This report was prepared from the three supplied project-phase PDFs. No external technical claims were added beyond the references already present in the Literature Review. Proposed design details are labeled as such; unknown implementation decisions remain open.